

Procesamiento de datos con PL/SQL

Clase en vivo 2 · 120 minutos

Programa: Arquitectura de Datos, Gobierno
y Orquestación — ARCA



**Escaneá el QR para registrar tu
asistencia**

Me presento...

1

Directora General en The eLabs: transformación digital e integración de IA.

2

Host del podcast Hack The Future: estrategia y liderazgo, no solo algoritmos.

3

Docente en la Maestría en Ciencia de Datos (Univ. Austral), ITBA y Di Tella.



Laura Sol Franco

CEO at The eLabs | Applied AI, Data, Risk & Future of Work



Apasionada por aplicar IA para mejorar la productividad y las decisiones.

Cinco capas y un gobierno

1 Fuentes

De dónde nace el dato: sistemas, archivos, APIs.

2 Ingesta

Cómo entra al ecosistema: por lotes o continua.

3 Almacenamiento

Dónde se guarda, en capas, hasta que se usa.

4 Procesamiento

Cómo se limpia, se transforma y se le aplican reglas. Es lo de hoy.

5 Consumo

Quién lo usa: tableros, alertas, analistas, modelos.

6 Gobierno

Responsabilidad, calidad y seguridad atraviesan todas las capas.

Hoy trabajamos la capa de procesamiento.

Hecho, dimensión, grano y decisión

1 Hecho

Lo que medimos: importes, cantidades, tiempos.

2 Dimensión

El contexto para cortar y filtrar: quién, qué, cuándo, dónde.

3 Grano

Qué representa una fila. Se declara primero: define qué se mide.

4 Registro de decisión

Problema, opciones, decisión, trade-off, control y señal para revisarla.

Hoy usamos la plantilla del registro de decisión para justificar una optimización.

Qué vamos a lograr hoy

PROPÓSITO

Implementar y revisar un procedimiento PL/SQL observable, transaccional y verificable.

RESULTADO OBSERVABLE

Cada equipo entrega un procedimiento almacenado con contrato, manejo de errores, prueba y decisión de optimización.



Recuperamos tres clases asincrónicas (M03-C01, C02 y C03) y las usamos para decidir y producir evidencia.

Agenda de la clase

Momento	Actividad	Evidencia
Encuadre	Cada equipo nombra una decisión difícil y el riesgo que intentó controlar	Hipótesis inicial escrita
Diagnóstico	Un caso incompleto: supuestos, dependencias y datos faltantes	Lista priorizada de preguntas
Demostración docente	Restricción, opción, trade-off, control y evidencia	Ejemplo anotado
Trabajo en equipos	Primera versión del procedimiento, con una alternativa descartada	Borrador verificable
Pausa y control cruzado	Otro equipo revisa el borrador con la checklist	Dos hallazgos y una pregunta
Iteración	Corregir, agregar controles y preparar la explicación oral	Versión revisada
Puesta en común	Dos o tres equipos presentan; el grupo pregunta	Defensa breve
Cierre	Decisión más sólida, riesgo abierto y próximo experimento	Ticket individual

SQL vs PL/SQL

Aspecto	SQL	PL/SQL
Qué es	Lenguaje declarativo para consultar y modificar datos	Extensión procedimental de SQL de Oracle
Cómo se piensa	Describís el resultado que querés	Describís los pasos y qué hacer ante un error
Unidad de trabajo	Una sentencia	Un bloque: DECLARE, BEGIN, EXCEPTION, END
Qué agrega	Filtrar, unir, agregar	Variables, condiciones, bucles y excepciones
Cuándo conviene	Consultar y transformar conjuntos	Cuando hace falta lógica con pasos, control de errores o reutilización

No compiten: PL/SQL envuelve sentencias SQL con lógica de control.

Ocho pasos para construir un procedimiento

1

Criterio de éxito primero

¿Qué consultas demostrarán que la corrida funcionó como esperabas?

2

Parámetros variables arriba

Identificador de carga, archivo y umbrales, como constantes o variables al tope del bloque.

3

%TYPE y %ROWTYPE

Anclar todas las variables de datos al esquema real.

4

Primero el camino esperado

Las ramas de manejo de error se agregan después.

5

Cada falla de negocio, con su respuesta

Monto fuera de rango o canal incorrecto: rechazo, corrección o detención.

6

Bloque anidado con EXCEPTION

Alrededor de la carga individual de cada fila.

7

Bitácora y un solo COMMIT

Al final, nunca dentro del bucle de filas.

8

Conciliación como validación final

Filas leídas = filas cargadas + rechazadas.

Cada paso tiene un lugar en el procedimiento de hoy.

Un contrato y quién confirma

EL CONTRATO, EN CINCO PARTES

1

Parámetros

IN, OUT e IN OUT, con tipos explícitos.

2

Precondiciones

Qué debe ser cierto antes de invocarlo.

3

Resultado

El estado que queda al terminar sin error.

4

Errores esperados

Cuáles se manejan y cuáles se propagan.

5

Efectos

Tablas modificadas y registros de auditoría.

¿DÓNDE VIVE EL COMMIT?

A · COMMIT dentro del procedimiento

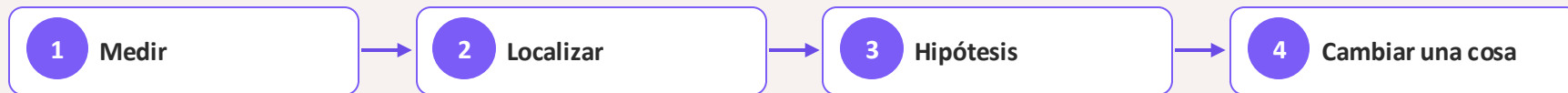
El invocador pierde el control: no puede revertir lo que ya se confirmó.

B · COMMIT en quien lo invoca

Mantiene el control: combina varios procedimientos en una sola unidad atómica.

El COMMIT debe vivir en el nivel más alto que tenga visibilidad completa de la operación.

Optimizar con evidencia



Síntoma	Evidencia a mirar	Causas posibles
Lento en general	Tiempo total y por etapa	Volumen creciente, recursos insuficientes
Lento en lectura	Plan de ejecución, estadísticas de tabla	Cardinalidad desactualizada, full scan innecesario
Lento en escritura	Esperas de I/O, contención de locks	Índices en destino, triggers, constraints diferidos
Lento al inicio	Tiempo de parseo, cursores compartidos	SQL dinámico sin bind variables, hard parse frecuente
Lento de forma intermitente	Eventos de espera, sesiones concurrentes	Contención de recursos, ventana de ejecución compartida

Optimizar es reducir incertidumbre con evidencia · medir primero · una variable a la vez · documentar siempre · depende del contexto

Una pregunta incómoda

¿Qué decisión de tu propuesta sería más costosa de revertir dentro de un año?

Costo

Seguridad

Calidad

Operación

Dependencia tecnológica

Cada equipo nombra una decisión difícil y el riesgo que intentó controlar. Una frase.

NexoServicios, con datos que faltan

Qué hay que integrar

Operaciones, CRM, archivos externos y una API.

Qué pide Dirección

Indicadores diarios.

Qué necesita Operaciones

Alertas durante el día.

Qué restricciones hay

Información sensible y un equipo pequeño.

Hasta siete preguntas antes de implementar

CINCO TEMAS OBLIGATORIOS: AL MENOS UNA PREGUNTA SOBRE CADA UNO

1

Volumen o latencia

Frecuencia y ventana de decisión.

2

Seguridad

Clasificación del dato.

3

Calidad

Un criterio medible, no solo «no hay nulos».

4

Recuperación

Reproceso, reintento y rollback.

5

Responsabilidad humana

Quién decide cuando algo falla.

DOS MÁS, A ELECCIÓN DEL EQUIPO

6

Pregunta 6

Por ejemplo: fuente de verdad, tolerancia a retrasos o reproceso.

7

Pregunta 7

Por ejemplo: propiedad del dato u observabilidad.

Una lista centrada solo en productos o tecnologías todavía no alcanza.

Cinco pasos, **en voz alta**

1

Restricción

Nombrá la restricción comprobable.

2

Opción

Compará dos opciones posibles.

3

Trade-off

Elegí una y declará el costo que aceptás.

4

Control

Definí un control que detecte el fallo.

5

Evidencia

Qué permite aprobar o rechazar.

Cada decisión de hoy sigue esta secuencia.

Del archivo a la tabla



Una falla de negocio afecta a UNA fila (se rechaza y sigue). Una falla de la corrida entera la detiene y la revierte.

La conciliación cierra la corrida: leídas = cargadas + rechazadas.

Qué entra, qué sale, qué falla

Parte	Qué es	Qué hace ante un problema
Entradas (IN)	Identificador de la carga y umbral de rechazo (por defecto, 20 %)	—
Salidas (OUT)	Filas leídas, cargadas y rechazadas	—
Falla de una fila	Una regla de negocio no se cumple	Se rechaza esa fila y la corrida sigue
Falla de la corrida	Más del 20 % del archivo se rechaza	Se detiene y se revierte todo
Falla inesperada	Cualquier otra cosa (OTHERS)	Se revierte y se propaga: no se silencia

Pasos 1 y 2 del procedimiento: el criterio de éxito y los parámetros arriba.

Constantes arriba, camino feliz primero

```
c_monto_min CONSTANT ordenes.monto%TYPE := 1;  
c_monto_max CONSTANT ordenes.monto%TYPE := 50000;  
v_canal      ordenes.canal%TYPE;  
  
FOR r IN (SELECT * FROM stg_ordenes  
          WHERE id_carga = p_id_carga  
          ORDER BY nro_fila) LOOP      -- r es %ROWTYPE  
    v_canal := UPPER(TRIM(r.canal));  
    INSERT INTO ordenes (...) VALUES (...);  
    p_cargadas := p_cargadas + 1;  
END LOOP;
```

1

Constantes arriba del bloque

2

%TYPE: tipos del esquema real

3

Primero el camino esperado

Pasos 2, 3 y 4. El código es PL/SQL especificado; el notebook ejecuta su equivalente.

Cada falla, con su respuesta

Falla	Respuesta	Por qué
Canal con espacios o minúsculas	Corrección	El dato es reconocible: se normaliza
Canal que no existe (FAX)	Rechazo	No hay forma segura de adivinar
Monto fuera de rango	Rechazo	Un monto imposible no se corrige solo
Ciente inexistente	Rechazo	Rompería la integridad de la base
Fecha inválida	Rechazo	No se puede ubicar la orden en el tiempo
Orden repetida o ya cargada	Rechazo	Duplicaría la facturación
Más del 20 % del archivo rechazado	Detención	El origen está mal: mejor frenar y revisar

Paso 5: cada falla tiene una respuesta explícita.

Una falla afecta solo a su fila

```
FOR r IN (...) LOOP
  BEGIN
    ... validar e insertar ...
  EXCEPTION
    WHEN e_negocio THEN -- rechazo: sigue
      INSERT INTO rechazos (...) VALUES (...);
  END;
END LOOP;

IF p_rechazadas > p_leidas * p_umbral_rechazo THEN
  RAISE e_umbral_superado; -- detiene la corrida
END IF;
COMMIT;
pr_bitacora(p_id_carga, 'OK', ...);
```

1 Bloque anidado por fila

2 El rechazo no frena el resto

3 Un solo COMMIT, al final

4 Bitácora autónoma

Pasos 6 y 7. La bitácora es una transacción autónoma: no se revierte con la carga.

Cierra la cuenta

5.000

filas leídas

4.725

filas cargadas

275

filas rechazadas

200

canales corregidos (se
cargaron)

$5.000 = 4.725 + 275$: la conciliación cierra, y cada rechazo tiene un motivo.

El original: ¿qué predecís?

```
CREATE OR REPLACE PROCEDURE
  actualizar_prioridad_clientes IS
  CURSOR c_clientes IS
    SELECT id_cliente, antiguedad_meses,
           tickets_abiertos
    FROM stg_clientes;
BEGIN
  FOR r IN c_clientes LOOP
    UPDATE dim_cliente
      SET score_prioridad =
          calcular_score(r.antiguedad_meses,
                        r.tickets_abiertos),
          fecha_actualizacion = SYSDATE
    WHERE id_cliente = r.id_cliente;
    COMMIT;
  END LOOP;
END actualizar_prioridad_clientes;
```

1

Si se cae en la fila 1.200,
¿cuántas filas quedan
actualizadas?

2

¿Cuántos COMMIT hace?

3

¿Qué le avisa a quien lo opera?

La vieron en la práctica asincrónica. Cada equipo anota sus respuestas antes de que las revelemos.

SQL masivo: ¿qué predecís?

```
MERGE INTO dim_cliente d
USING (
  SELECT id_cliente,
    CASE
      WHEN tickets_abiertos >= 5 THEN 100
      WHEN antiguedad_meses >= 24 THEN 70
      ELSE 40
    END AS score_calculado
  FROM stg_clientes
) s ON (d.id_cliente = s.id_cliente)
WHEN MATCHED THEN
  UPDATE SET d.score_prioridad = s.score_calculado,
    d.fecha_actualizacion = SYSDATE;
v_filas := SQL%ROWCOUNT;
COMMIT;
INSERT INTO log_procesos (...) VALUES (...);
COMMIT;
```

1

¿Dónde vive la regla del score?

2

¿Qué pasa con un cliente que está en stg y no en dim?

3

¿Por qué hay dos COMMIT y qué riesgo tienen?

Código resumido para que entre en el slide. Respondé antes de que lo revelemos.

Por lotes: ¿qué predecís?

```
OPEN c_clientes;
LOOP
  FETCH c_clientes BULK COLLECT
    INTO v_ids, v_ant, v_tick LIMIT c_lote;
  EXIT WHEN v_ids.COUNT = 0;
  FOR i IN 1 .. v_ids.COUNT LOOP
    v_scores(i) := calcular_score(...);
  END LOOP;
  BEGIN
    FORALL i IN 1 .. v_ids.COUNT SAVE EXCEPTIONS
      UPDATE dim_cliente SET ... WHERE ...;
  EXCEPTION
    WHEN OTHERS THEN
      ... registra SQL%BULK_EXCEPTIONS ...
  END;
END LOOP;
CLOSE c_clientes; COMMIT;
```

1

Si falla una fila de un lote, ¿qué queda en el log?

2

¿Y si el error no es de FORALL?

3

¿Qué número no te devuelve?

Código resumido para que entre en el slide. Respondé antes de que lo revelemos.

Una variable a la vez

Variante	Llamadas a la base	COMMIT	Tiempo	Contra la 1
1 · original (COMMIT por fila)	6.061	2.020	1,7 s	referencia
1b · el mismo, con un solo COMMIT	2.023	1	0,013 s	unas 135 veces más rápida
2 · SQL masivo	4	1	0,004 s	unas 485 veces más rápida
3 · por lotes	14	1	0,009 s	unas 185 veces más rápida

Las cuatro dejan el mismo resultado. Los tiempos son de una corrida; los conteos no dependen de la máquina.

La decisión de optimización, por escrito

CONTEXTO Y DECISIÓN

- Problema que se resuelve
- Decisión que habilita
- Restricciones verificadas
- Supuestos por confirmar

OPCIONES

- Opción A: beneficio, costo o riesgo, evidencia necesaria
- Opción B: lo mismo
- Opción C: lo mismo

DECISIÓN ELEGIDA

- Elección
- Trade-off aceptado
- Control: cómo se detecta que falla
- Señal para revisar esta decisión
- Responsable de revisión

Cada equipo la completa para la versión que elige, con una alternativa descartada.

Tres versiones, una decisión

1

Comparar

Completá la tabla de seis preguntas con las tres versiones.

2

Decidir

Elegí una para el proceso nocturno y completá el registro de decisión.

3

Asegurar

Escribí su contrato en cinco partes y una prueba de borde.

Entregable: tabla, registro de decisión, contrato y prueba.

Seis preguntas, tres versiones

	1 · Original	2 · SQL masivo	3 · Por lotes
Unidad de trabajo			
Dónde está el COMMIT y qué pasa si falla a mitad			
Cómo maneja los errores			
Qué queda registrado de la corrida			
Dónde vive la regla del score			
Se puede volver a ejecutar sin daño			

Ninguna versión tiene todavía un contrato escrito: eso también cuenta.

La entrega responde seis preguntas

Problema

¿Qué problema y qué decisión resuelve?

Alcance

¿Qué entra y qué queda fuera de alcance?

Alternativa

¿Qué se descartó y por qué?

Prueba

¿Cómo se prueba antes de pasar a producción?

Recuperación

¿Cómo se detecta, contiene y recupera una falla?

Responsable

¿Quién decide cuando un control no pasa?

Errores a observar

Productos sin flujos

Dibujar herramientas sin responsabilidades ni flujos.

Monitoreo no es recuperación

Detectar no alcanza: hay que definir qué hacer.

Calidad es más que «no hay nullos»

Necesita un criterio medible.

Decisiones humanas escondidas

No se ocultan detrás de una automatización.

Supuestos sin declarar

Disponibilidad, volumen o permisos que el caso no define.

Herramienta antes que requisito

Primero la restricción, después el producto.

Cuatro niveles de devolución

CUATRO NIVELES DE DEVOLUCIÓN

Insuficiente

No conecta el problema con el diseño.

Inicial

Es técnicamente posible, pero no explica cómo se valida o recupera.

Sólido

Conecta problema, diseño y pruebas, y declara sus trade-offs.

Defendible

Además, resiste preguntas sobre fallas, costo y recuperación.

SE EVALÚA

Trazabilidad entre problema y diseño · manejo de trade-offs · verificabilidad · operación segura · claridad de la defensa.

Otro equipo revisa con la checklist

- ☐ El problema está formulado como decisión y no como deseo.
- ☐ Las fuentes, transformaciones y consumidores son identificables.
- ☐ Los límites de seguridad y responsabilidad están visibles.
- ☐ Existe un criterio de calidad medible.
- ☐ Hay una estrategia de reintento, reproceso o rollback.
- ☐ La propuesta registra evidencia suficiente para explicar qué ocurrió.
- ☐ El costo o complejidad aceptados están declarados.

Cada equipo devuelve dos hallazgos y una pregunta.

Aceptar o rechazar, **con justificación**

Aceptá o rechazá

Cada observación recibida.

Justificá

No alcanza con «corregido».

Agregá controles

Para cada falla que apareció.

Prepará la explicación oral

Para la puesta en común.

Cinco preguntas de debrief

1

Latencia

¿Qué decisión cambiaría si la latencia requerida se redujera diez veces?

2

Radio de impacto

¿Qué componente concentra el mayor radio de impacto ante una falla?

3

Evidencia

¿Qué evidencia necesitaría una persona ajena al equipo para reconstruir lo ocurrido?

4

Falsos positivos

¿Qué control podría generar falsos positivos y cómo se administraría?

5

Dependencias

¿Qué parte del diseño depende de una política o infraestructura todavía no confirmada?

Qué me llevo y qué dejo escrito

CUATRO IDEAS PARA LLEVARSE

1

Primero el criterio de éxito

Qué consultas demuestran que la corrida funcionó.

2

Un contrato, un solo COMMIT

Confirmar en el nivel que ve toda la operación.

3

Los errores no se silencian

Se rechazan, se registran o se propagan.

4

Optimizar con evidencia

Medir, localizar y cambiar una sola cosa.

TICKET DE SALIDA · UNA FRASE POR CADA UNO

Decisión más sólida

La que mejor puede defender tu equipo.

Riesgo abierto

El que todavía no está controlado.

Próximo experimento

Qué probarías primero.

Qué decisión voy a defender, qué riesgo queda abierto y qué voy a probar primero.

Hacia la clase 3

Un procedimiento procesa datos que ya están en la base. ¿Qué cambia cuando el dato llega de archivos, de una API o de otra base?

Procesadores

Conexiones

Flujo de datos

Trazabilidad

Eso introduce la clase 3: construcción de pipelines con Apache NiFi.

PREGUNTAS Y RESPUESTAS

Q&A

¿Qué dudas, casos o ideas quedaron?

¡Muchas gracias!

Laura Sol Franco · Digital House

Programa: Arquitectura de Datos, Gobierno y Orquestación — ARCA